

📄

HUANGLIZI / LViT

Public

<> Code

🔍 Issues 6

🔗 Pull requests 1

🔄 Actions

📁 Projects

⋮

🔗 main ▾

🔗 Branches

📁 Tags

🔗

📁

Go to file

<> Code ▾

⋮

🕒

📁 IMG

📁 LV\_loss

📁 datasets/MoNuSeg

📁 nets

📄 Config.py

📄 LICENSE

📄 Load\_Dataset.py

📄 README.md

📄 Train\_one\_epoch.py

📄 requirements.txt

📄 test\_model.py

📄 train\_model.py

📄 utils.py

📖 README

📄 MIT license

LViT

This repo is the official implementation of "LViT: Language meets Vision Transformer in Medical Image Segmentation" [Arxiv](#), [ResearchGate](#), [IEEEExplore](#)

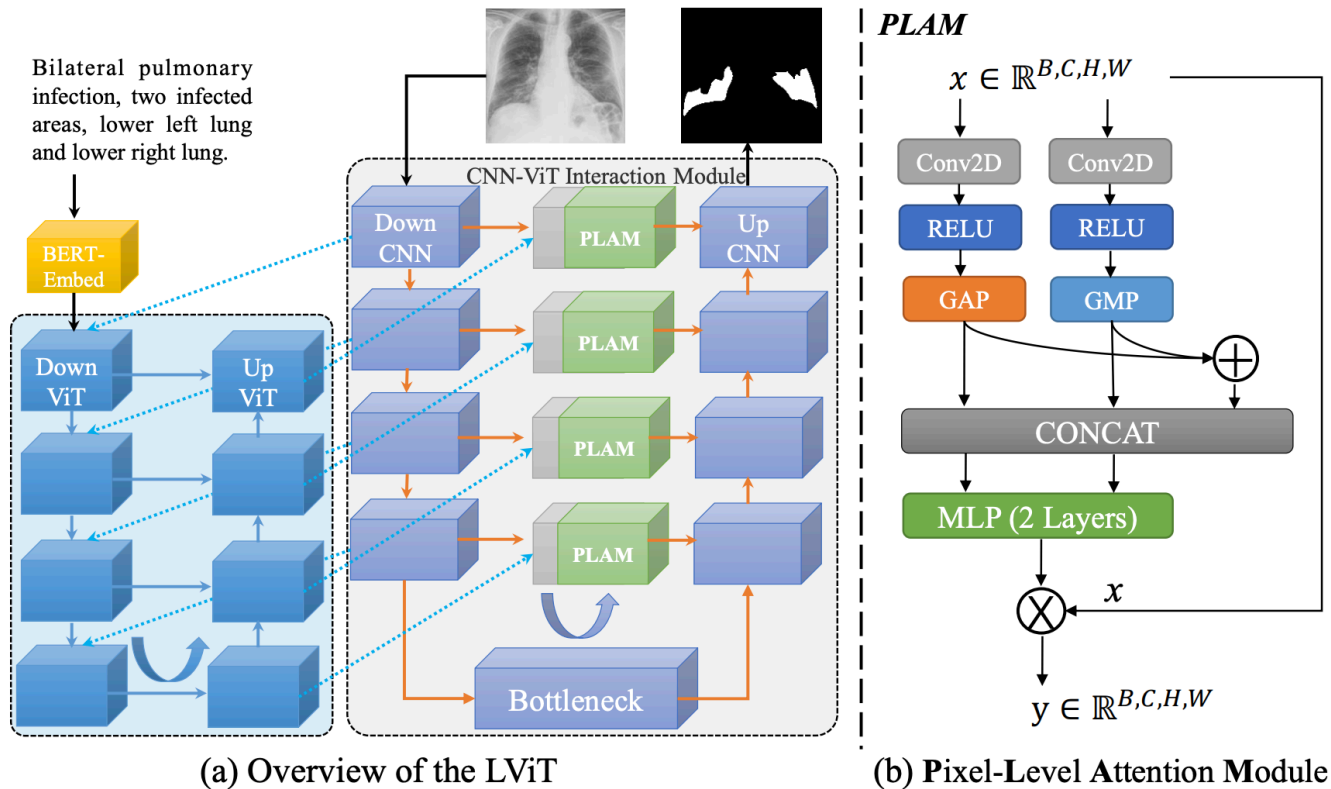


Figure 2: Illustration of (a) the proposed LViT model, and (b) PLAM. The proposed LViT model is a Double-U structure formed by combining a U-shape CNN branch with a U-shaped ViT branch.

## Requirements

Python == 3.7 and install from the `requirements.txt` using:

```
pip install -r requirements.txt
```

Questions about NumPy version conflict. The NumPy version we use is 1.17.5. We can install bert-embedding first, and install NumPy then.

## Usage

### 1. Data Preparation

#### 1.1. QaTa-COV19, MosMedData+ and MoNuSeg Datasets (demo dataset)

The original data can be downloaded in following links:

- QaTa-COV19 Dataset - [Link \(Original\)](#)
- MosMedData+ Dataset - [Link \(Original\)](#) or [Kaggle](#)
- MoNuSeg Dataset (demo dataset) - [Link \(Original\)](#)
- ESO-CT Dataset [1] [2]

[1] Jin, Dakai, et al. "DeepTarget: Gross tumor and clinical target volume segmentation in esophageal cancer radiotherapy." *Medical Image Analysis* 68 (2021): 101909.

[2] Ye, Xianghua, et al. "Multi-institutional validation of two-streamed deep learning method for automated delineation of esophageal gross tumor volume using planning CT and FDG-PET/CT." *Frontiers in Oncology* 11 (2022): 785788.

The text annotation of QaTa-COV19 has been released!

(Note: The text annotation of QaTa-COV19 train and val datasets [download link](#). The partition of train set and val set of QaTa-COV19 dataset [download link](#). The text annotation of QaTa-COV19 test dataset [download link](#).)

(Note: The contrastive label is available in the repo.)

(Note: The text annotation of MosMedData+ train dataset [download link](#). The text annotation of MosMedData+ val dataset [download link](#). The text annotation of MosMedData+ test dataset [download link](#).)

If you use the datasets provided by us, please cite the LViT.

## 1.2. Format Preparation

Then prepare the datasets in the following format for easy use of the code:

```
├─ datasets
│   ├── QaTa-Covid19
│   │   ├── Test_Folder
│   │   │   ├── Test_text.xlsx
│   │   │   ├── img
│   │   │   └── labelcol
│   │   ├── Train_Folder
│   │   │   ├── Train_text.xlsx
│   │   │   ├── img
│   │   │   └── labelcol
│   │   └── Val_Folder
│   │       ├── Val_text.xlsx
│   │       ├── img
│   │       └── labelcol
│   └── MosMedDataPlus
│       ├── Test_Folder
│       │   ├── Test_text.xlsx
│       │   ├── img
│       │   └── labelcol
│       ├── Train_Folder
│       │   ├── Train_text.xlsx
│       │   ├── img
│       │   └── labelcol
│       └── Val_Folder
│           ├── Val_text.xlsx
│           ├── img
│           └── labelcol
```

## 2. Training

### 2.1. Pre-training

You can replace LViT with U-Net for pre training and run:

```
python train_model.py
```

### 2.2. Training

You can train to get your own model. It should be noted that using the pre-trained model in the step 2.1 will get better performance or you can simply change the model\_name from LViT to LViT\_pretrain in config.

```
python train_model.py
```

## 3. Evaluation

### 3.1. Test the Model and Visualize the Segmentation Results

First, change the session name in `Config.py` as the training phase. Then run:

```
python test_model.py
```

You can get the Dice and IoU scores and the visualization results.

4. Results

| Dataset     | Model Name          | Dice (%) | IoU (%) |
|-------------|---------------------|----------|---------|
| QaTa-COV19  | U-Net               | 79.02    | 69.46   |
| QaTa-COV19  | LViT-T              | 83.66    | 75.11   |
| MosMedData+ | U-Net               | 64.60    | 50.73   |
| MosMedData+ | LViT-T              | 74.57    | 61.33   |
| MoNuSeg     | U-Net               | 76.45    | 62.86   |
| MoNuSeg     | LViT-T              | 80.36    | 67.31   |
| MoNuSeg     | LViT-T w/o pretrain | 79.98    | 66.83   |

4.1. More Results on other datasets

| Dataset                   | Model Name | Dice (%) | IoU (%) |
|---------------------------|------------|----------|---------|
| <a href="#">BKAI-Poly</a> | LViT-TW    | 92.07    | 80.93   |
| ESO-CT                    | LViT-TW    | 68.27    | 57.02   |

5. Reproducibility

In our code, we carefully set the random seed and set cudnn as 'deterministic' mode to eliminate the randomness. However, there still exist some factors which may cause different training results, e.g., the cuda version, GPU types, the number of GPUs and etc. The GPU used in our experiments is 2-card NVIDIA V100 (32G) and the cuda version is 11.2. And the upsampling operation has big problems with randomness for multi-GPU cases. See <https://pytorch.org/docs/stable/notes/randomness.html> for more details.

Reference

Releases

No releases published

Packages

Contributors

Languages

Python 100.0%